Checklist: revisione di sicurezza di codice scritto con un agente

Un foglio da tenere accanto al terminale quando si rivede un ramo su cui ha lavorato un agente. Vale per la palestra e per il vostro progetto. Gli esempi di comando sono per un progetto Node.js con git; per altri linguaggi cambiano i nomi dei file, non le domande.

Regola di partenza: ogni problema che segnalate ha una prova, cioè un comando o una richiesta che lo mostra. Senza prova è un'ipotesi, e va scritta come tale.

1. Segreti

- Ci sono chiavi, password, token o indirizzi con credenziali nei file? Anche "di esempio"? `git grep -n -i -E "chiave|password|passwd|token|secret|api[_-]?key"`
- I file di configurazione con valori veri sono esclusi da git? Controllate `.gitignore` e `git ls-files`.
- Qualcosa stampa la configurazione o le variabili d'ambiente in un log o nella console? `git grep -n -E "console\.log|print\(" -- "*.js" "*.py"`
- La cronologia contiene segreti che oggi non si vedono più? `git log --all -p -S "testo-da-cercare" --oneline`
- Se un segreto vero è finito in un commit: prima si revoca o si cambia, poi si pulisce la cronologia.

2. Dipendenze

- Che cosa è cambiato in `package.json` e nel lockfile rispetto alla partenza? `git diff main -- package.json package-lock.json`
- Per ogni dipendenza: dove si usa (`git grep -n "require('nome')"`) e che cosa si farebbe senza. Il linguaggio lo fa già?
- Il nome è quello giusto? Cercatelo a mano sul registro (per Node: npmjs.com), con autore, data e numero di download. Diffidate dei nomi quasi uguali a quelli noti.
- Il codice funziona anche senza la libreria? Allora la libreria forse non serve.
- Si installa dal lockfile (`npm ci`), non con un `npm install` che lo riscrive.

3. Input che arriva a file, comandi e database

- Seguite ogni valore che arriva da fuori (parametri dell'URL, corpo della richiesta, argomenti della riga di comando, file caricati) fino a dove viene usato. `git grep -n -E "searchParams|req\.body|process\.argv" git grep -n -E "readFile|writeFile|exec|spawn|query\("`
- Un valore esterno diventa un percorso di file? Allora: che cosa succede con `..` o con un percorso assoluto? Il file finale resta nella cartella prevista?
- Un valore esterno finisce in un comando di shell o in una query? Si passa come parametro, mai incollato nella stringa.
- Si accetta solo il formato atteso (elenco di valori ammessi o espressione regolare), invece di provare a ripulire l'input.
- Provatelo sul vostro PC, con il server che ascolta solo su `127.0.0.1`.

4. Istruzioni nascoste in ciò che l'agente legge

- Ci sono commenti HTML, testo nascosto o frasi rivolte a "l'assistente" o "l'IA" in documenti, dati, issue o pagine che l'agente ha letto? `git grep -n "<!--" git grep -n -i -E "assistent|intelligenza artificiale|ignora"`
- L'agente ha chiesto di fare qualcosa che non c'entrava con la richiesta? Ha creato file che nessuno ha chiesto? `git status`
- Leggete il file sorgente, non solo l'anteprima: un commento HTML nell'anteprima del Markdown non si vede.

5. Comandi distruttivi e permessi

- Negli script e nelle istruzioni del progetto ci sono `rm -rf`, `git reset --hard`, `git push --force`, `git clean`, cancellazioni di tabelle?
- Il repository di esercizio ha un remote? Deve restare vuoto: `git remote -v`.
- Avete approvato qualcosa con "non chiedere più"? Controllate le regole salvate con `/permissions` in Claude Code.
- Si lavora su un ramo, partito da un commit pulito.

6. Prima di dichiarare finito

- Ogni problema ha file, riga, prova e gravità.
- Ogni correzione si verifica con la stessa prova che ha mostrato il problema, poi con i test.
- I test verdi non dicono niente su ciò che non provano.
- Scrivete che cosa non avete controllato.

Codice e dati del corso sono inventati. Questa checklist non sostituisce una revisione di sicurezza professionale su un sistema in produzione.
